Full NTT ASIC RTL handoff summary

Status

The project now contains a full 256-point NTT ASIC-compatible RTL handoff top with a generic SRAM macro-wrapper boundary.

Recommended handoff top:

- ‘ntt_core_256_sram‘

This supersedes:

- ‘ntt_core_8‘ ? earlier 8-point prototype
- ‘ntt_core_256‘ ? full 256-point top using direct inferred/register-array storage

Original FPGA/HLS context

The original project used Vitis/Vivado/HLS-style implementation files with FPGA-oriented constructs such as HLS pragmas, AXI interface inference, BRAM binding directives, and generated implementation artifacts.

Those files remain useful as algorithm/interface reference material, but the clean ASIC handoff path is the hand-written RTL under:

- ‘rtl/‘

The OpenLane owner should not use generated FPGA/HLS implementation files under ‘hls-design/‘ as the physical-design source unless deliberately comparing against the old implementation.

Clean ASIC-oriented RTL created

Arithmetic/datapath RTL:

- ‘rtl/mod_add.sv‘
- ‘rtl/mod_sub.sv‘
- ‘rtl/modmul_montgomery.sv‘
- ‘rtl/ntt_butterfly.sv‘

SRAM wrapper:

- ‘rtl/asic_sram_1rw.sv‘

Full target RTL:

- ‘rtl/ntt_core_256_sram.sv‘

Bring-up/reference RTL retained:

- ‘rtl/ntt_core_256.sv‘
- ‘rtl/simple_dual_port_ram.sv‘

- ‘rtl/ntt_stage_2.sv‘
- ‘rtl/ntt_stage_4.sv‘
- ‘rtl/ntt_stage_8.sv‘
- ‘rtl/ntt_core_8.sv‘
- ‘rtl/ntt_stage_8_registered.sv‘

Use ‘ntt_core_256_sram‘ for the full-design handoff.

What ‘ntt_core_256_sram‘ implements

‘ntt_core_256_sram‘ implements a fixed 256-point HLS-style NTT schedule with:

- ‘N = 256‘
- ‘Q = 8380417‘
- ‘Q_INV = 4236238847‘
- load interface for 256 input data words
- load interface for 256 bit-reversal table entries
- load interface for 255 twiddle values
- ‘start‘, ‘busy‘, and ‘done‘ control
- readback interface for 256 result words
- one shared butterfly datapath
- synchronous SRAM-wrapper storage
- ping/pong SRAM-wrapper memories for stage results
- 8 NTT stages
- 128 butterflies per stage

The controller performs:

1. Input permutation into ping storage using the bit-reversal table.
2. Stage iteration from length 1 through length 128.
3. HLS-style flattened butterfly index generation.
4. Twiddle lookup using the stage offset and butterfly-local index.
5. SRAM-style reads for ‘u‘, ‘v‘, and twiddle data.
6. Butterfly execution through the verified modular multiplier/add/sub datapath.
7. SRAM-style writes into the inactive ping/pong result memory.
8. Final result readback through ‘read_addr‘ and ‘read_data‘.

SRAM macro boundary

The design now uses:

- ‘rtl/asic_sram_1rw.sv‘

as a generic 1-read/write SRAM wrapper and behavioral simulation model.

This wrapper is the intended macro-binding boundary. Because no exact PDK/SRAM macro library was specified, the RTL does not hard-code a Sky130/GF180/etc. macro name.

For a real tapeout flow, the physical-design owner should bind or replace ‘asic_sram_1rw‘ with process-specific SRAM macro instances and provide the matching:

- Verilog model

- Liberty ‘.lib‘
- LEF abstract
- GDS layout
- OpenLane/PDK setup files

Verification completed

The following checks passed locally:

PASS: tb_ntt_core_256_sram
PASS: ntt_core_256_sram Yosys synthesizability check completed
PASS: SRAM-backed full NTT OpenLane handoff readiness checks completed

The readiness script is:

- ‘scripts/check_openlane_handoff_readiness.sh‘

It runs:

- ‘scripts/run_ntt_core_256_sram_tb.sh‘
- ‘scripts/lint_rtl.sh‘
- ‘scripts/run_synth_ntt_core_256_sram.sh‘

Golden model and vectors

The full 256-point RTL testbench uses the Python golden model:

- ‘sim/ntt_golden.py‘

Generated vectors:

- ‘sim/test_vectors/input_256.hex‘
- ‘sim/test_vectors/bitrev_256.hex‘
- ‘sim/test_vectors/twiddles_256.hex‘
- ‘sim/test_vectors/expected_256.hex‘
- ‘sim/test_vectors/params_256.txt‘

The testbench is:

- ‘sim/tb_ntt_core_256_sram.sv‘

Handoff support files

Simulation/test:

- ‘scripts/run_ntt_core_256_sram_tb.sh‘

Lint/synthesis:

- ‘scripts/lint_rtl.sh‘
- ‘scripts/synth_ntt_core_256_sram_yosys.tcl‘
- ‘scripts/run_synth_ntt_core_256_sram.sh‘

- ‘build/synth_ntt_core_256_sram_yosys.log‘

OpenLane starter files:

- ‘constraints/ntt_core_256_sram.sdc‘
- ‘openlane/ntt_core_256_sram/config.tcl‘
- ‘docs/openlane_handoff.md‘

BRAM/DRAM/SRAM status

The clean handoff RTL does not instantiate known FPGA BRAM primitives such as:

- ‘RAMB18E1‘
- ‘RAMB36E1‘
- ‘xpm_memory_*‘
- Vivado block-memory IP

The design does not use DRAM. It now uses a generic SRAM wrapper boundary.

The wrapper must still be connected to real PDK-specific SRAM macro views before production tapeout.

What remains before real tapeout

The project is ready for SRAM-aware full-design OpenLane physical-flow exploration, but not automatic manufacturing signoff.

Remaining work for the physical-design owner:

1. Select exact PDK, standard-cell library, and SRAM macro library.
2. Bind ‘asic_sram_1rw‘ to real SRAM macros or replace it with macro-specific wrappers.
3. Add SRAM macro ‘.lib‘, ‘.lef‘, ‘.gds‘, and Verilog model views.
4. Run technology-mapped synthesis.
5. Review timing, especially around the Montgomery multiplier and SRAM access paths.
6. Pipeline or restructure datapath if timing requires it.
7. Tune floorplan, SRAM macro placement, utilization, IO constraints, and clock period.
8. Run placement, CTS, routing, DRC, LVS, antenna checks, extraction, STA, and signoff checks.
9. Generate and review final GDSII only after the above checks pass.

Correct handoff statement

Use this wording:

"Full 256-point NTT ASIC-compatible RTL with generic SRAM macro-wrapper boundary and OpenLane starter handoff package."

Do not describe it as:

"Final tapeout-ready GDSII."